جامعة أبوظبي
Abu Dhabi University

CEN454 Individual reflection Report

Baggage Threat Detection and Localization

Student 1: Yasmin Magdy Loksha 1087999
Faculty: Dr. Sajid Gul Khawajah

Date: October 4, 2026

Abstract

This project presents a computer vision-based baggage threat detection and localization system for identifying dangerous objects in X-ray baggage images, including guns, knives, and shurikens. The project combines classical image processing, machine learning, and deep learning techniques to build a complete detection pipeline suitable for noisy and complex baggage images. My primary contribution focused on image preprocessing, localization, segmentation evaluation, and debugging of the U-Net-based localization workflow. Initially, traditional segmentation techniques such as thresholding, edge detection, Otsu's method, and morphological operations were explored; however, these methods failed to generalize effectively under complex backgrounds and overlapping object conditions. Consequently, the localization stage was redesigned using a binary U-Net segmentation architecture trained using annotated masks. Multiple preprocessing pipelines, filtering methods, post-processing techniques, and evaluation metrics were tested iteratively to improve localization performance. Dice score and Intersection over Union (IoU) were used as the primary evaluation metrics because pixel-level accuracy alone proved misleading due to background dominance in segmentation images. The final system successfully demonstrated improved localization consistency compared to traditional image-processing approaches while also highlighting the importance of annotation quality, preprocessing selection, and iterative debugging in machine learning-based computer vision systems.

Contents

1 Brief Project Context and My Personal Role 3

2 Personal Contribution 3

3 Engineering Process and Design Decisions 5

4 Technical Challenges and Problem Solving 8

5 Iterations, Testing, and Refinement 9

6 Technical Reasoning Behind Final Choices 10

7 Lessons Learned 11

8 Possible Improvements and Future Work 11

9 Transparent AI Usage 12

10 Walkthrough Video Link 12

1 Brief Project Context and My Personal Role

This project aims to develop a computer vision system for detecting baggage threats in X-ray images. Objects to be detected are guns, knives, and shurikens, in addition to safe baggage images that don't include any of these objects for the sake of classification. The full pipeline of the project is shown in Figure 1. Technically, the goal was not a fully working system output, but to apply our theoretical knowledge from the course to explore how we can integrate noise removal, segmentation, localization, image enhancement, and classification into a full pipeline suitable for noisy baggage X-ray images.

My personal contribution mainly focused on image preprocessing, localization, evaluation of segmented output, and debugging. I worked on exploring the different types of noise that affected the images, and I tested some filters to remove this noise, in addition to training and testing the U-Net localizer using the given annotation masks. Then, after training, I compared the predicted masks with the ground truth annotations, and I refined the post-processing used to convert masks into bounding boxes.

Throughout the project, I didn't follow a single approach from the beginning to the end of the project. My approach has gone through countless iterations. Initially, I was exploring traditional types of segmentation using techniques such as thresholding, edge detection, Otsu's method, morphological operations, etc. Once those solutions did not yield satisfactory results for either complex images and/or cases where multiple objects needed to be identified, I moved from a traditional method toward a U-Net-based localization system while keeping classification as a traditional computer vision and machine learning method vision/ml method. After trying multiple models based on U-Net architecture and finding that none had good generalization properties, I began exploring issues with my annotation formats; class imbalances; binary versus multi-class segmentation; post-processing techniques; and use of Google Colab for training.

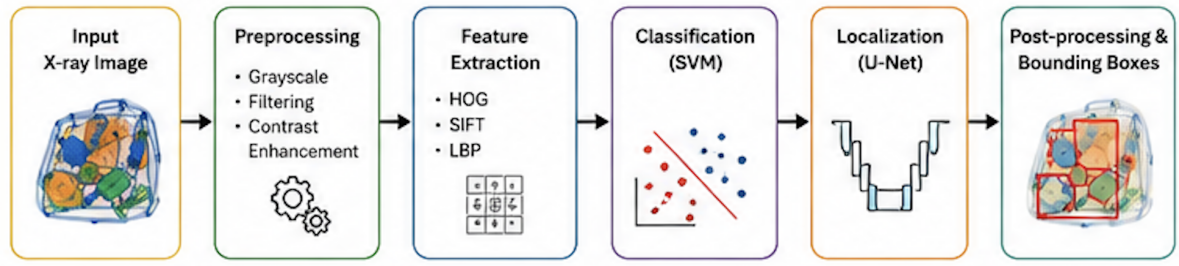


Figure 1: Overall Pipeline of the baggage threat detection and localization system

2 Personal Contribution

My contribution to this work involved developing and testing a localization processing pipeline. I created the MATLAB preprocessing functions of converting images to gray scale, resizing images, and applying median filtering, Wiener filtering, and contrast enhancement. I tested how each form of preprocessing affected segmentation results and later realized that aggressive forms of contrast enhancement could make it more difficult for the U-Net to detect dark objects that were not a threat; they would instead enhance the background noise or artifacts.

I also developed the U-Net training workflow in MATLAB; the U-Net training workflow requires the pairing of each image with its matching annotation mask, as well as building image and mask data stores, pre-processing the masks, training the localizing network (U-Net), saving the trained U-Net, and creating a predicted mask for testing purposes. During this workflow stage, I experienced numerous issues, such as no image/mask pairing, incorrect folder path issues, case sensitivity of folder naming conventions, incorrect datastore formatting, and the

absence of the preprocessing functions needed to complete the workflow.

One of my other major contributions involved validating the predicted masks against the ground-truth annotation masks. I created and ran a code that showed the original image, predicted mask, ground truth annotation mask, and localization result. I also calculated the IoU and Dice coefficient for the segmentation to evaluate how well the segmentation performed. During this process, I found that many of the wrong zero IoU values were not due to an issue with the model but were due to mistakes made during the evaluation phase, for example, comparing the ground truth mask in the form of 0/255 vs. the predicted mask of 0/1.

Another contribution I made included facilitating the transition from training in MATLAB only to training using Google Colab. I investigated why training on MATLAB was taking a while on my MacBook M4, evaluated the limitations of using the CPU for training, and transitioned the training for U-net into PyTorch from within Google Colab, after discussing it with my instructor. I completed the process for loading data to the Colab environment, generating binary masks from training data, implementing the U-net model, implementing the Dice + BCE loss function, augmentation, validation metrics, and visualizing predictions. This enabled much faster experimentation and had greater control over the training process. After a few validation iterations, some images show zero overlap. My colleague Shahd noticed that the ground truth masks are not correct or do not match the original image, so this led to reporting this to the doctor for further confirmation. A sample of these images is shown in Figure 2.

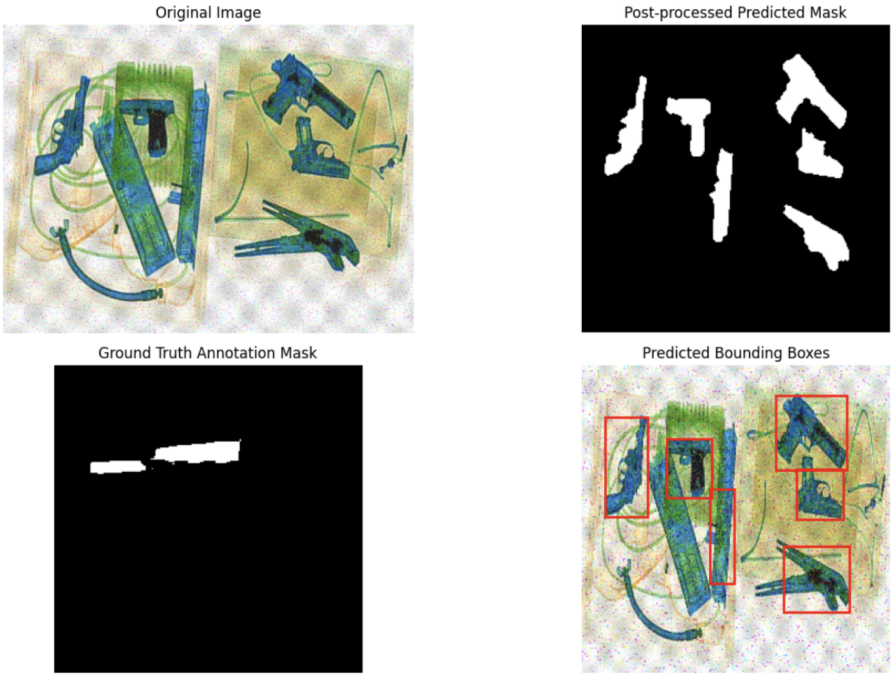


Figure 2: Example of an incorrect ground truth annotation mask that produced misleading localization evaluation results.

3 Engineering Process and Design Decisions

In my early attempts at threat localization, I thought that simple to moderate classical image-processing workflows would be a solution for the problem. Traditional segmentation approaches, including thresholding, edge detection, and morphological refinement, were initially explored for threat localization. The performance of each method was analyzed based on segmentation continuity, robustness against cluttered baggage backgrounds, and localization consistency. However, these methods produced fragmented masks, incomplete object boundaries, and unstable localization performance under cluttered baggage backgrounds and overlapping objects, as shown in Figure 3. Consequently, a U-Net-based deep learning localization model was adopted to achieve more robust and consistent segmentation performance.

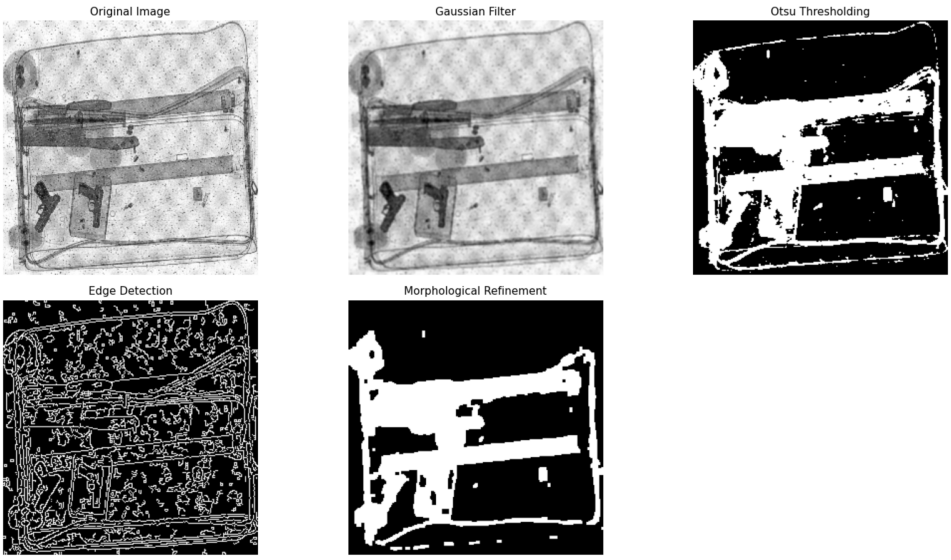


Figure 3: Failure cases of traditional segmentation approaches, including Otsu thresholding and edge detection, under complex X-ray baggage backgrounds, motivating the transition to U-Net-based localization.

To better analyze the limitations of each segmentation approach, a comparative evaluation was conducted based on localization consistency and robustness under cluttered baggage conditions.

Table 1: Comparison of Traditional Segmentation Methods

Method	**Observed Result**
Otsu Thresholding	Produced unstable segmentation under cluttered X-ray backgrounds and varying object intensities.
Edge Detection	Generated fragmented object boundaries and incomplete threat localization results.
Morphological Operations	Improved mask continuity and reduced small noisy regions after thresholding.
U-Net Segmentation	Achieved the most robust and consistent localization performance across complex baggage images.

Based on the comparative analysis, traditional segmentation methods alone were insuffi-

cient for reliable localization under complex baggage conditions. Consequently, a U-Net-based deep learning localization model was adopted to improve segmentation robustness and object boundary consistency.

Subsequently, I began moving towards building a U-Net-based localizer because my annotated masks associated with the project could provide pixel-level supervised learning for the U-Net model, allowing it to learn object localization more effectively than manually tuned thresholding methods.

I initially trained a multiclass U-Net to recognize the classes: background, gun, knife, and shuriken (four total classes). The preprocessing pipeline used for the U-Net localization model was intentionally kept lightweight to preserve important threat boundaries and structural details within X-ray baggage images. First, all grayscale images were resized to (256 x 256) pixels to maintain a consistent input size for the network. Mild Gaussian filtering using a (3 x 3) kernel was then applied to reduce small-scale noise while preserving the main object contours. Finally, pixel intensities were normalized to the range ([0,1]) to stabilize neural network training and improve numerical convergence.

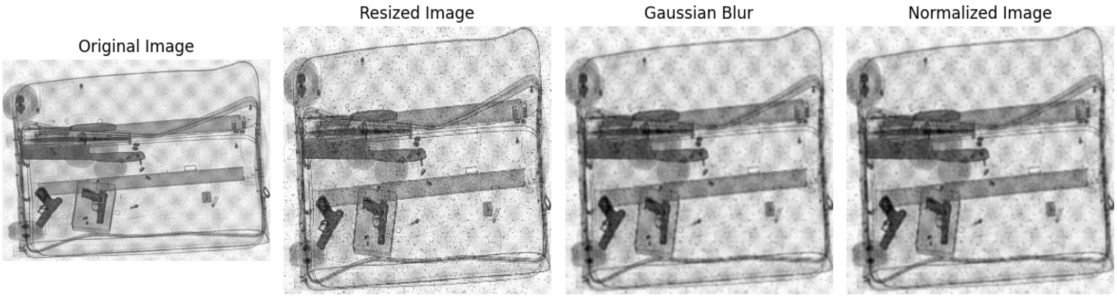


Figure 4: Preprocessing stages applied before U-Net localization, including image resizing, Gaussian denoising, and normalization.

X-ray baggage images primarily rely on structural and intensity information rather than natural RGB color representation; therefore, grayscale preprocessing reduced computational complexity while preserving important object boundaries.

This initially appeared to be a reasonable approach because a single U-Net model could theoretically perform both localization and classification simultaneously. However, during the evaluation phase, the model struggled to produce reliable class predictions and frequently generated false blobs or failed to localize objects correctly when multiple threat classes appeared within the same image. At this point, I realized that the annotation structure of the dataset itself created significant challenges for multiclass segmentation because several images within the same class folder contained multiple threat categories simultaneously.

In light of these findings, the localization design was improved by adopting binary U-Net localization (background versus object), thereby separating localization from classification. The final U-Net output layer used sigmoid activation to generate probability masks, followed by thresholding to obtain binary segmentation outputs. In this architecture, the U-Net model only answers the question: "Where is the threat object?" The classification task could then be handled separately using classical machine learning techniques. This separation aligned with the project constraints because Convolutional Neural Networks (CNNs) were utilized only for localization and not for classification.

The decision between semantic masks and bounding-box localization was also carefully considered. Initially, semantic segmentation was preferred because it provides more detailed object boundaries. However, bounding boxes extracted from the predicted masks were ultimately selected because they are easier to visualize, compare, and interpret during localization evaluation. The bounding boxes were not directly generated by an object detector; instead, they

were extracted from connected components obtained from the post-processed U-Net prediction masks.

The final localization pipeline successfully detected and localized multiple threat objects under complex baggage conditions. After U-Net prediction, morphological refinement and connected-component filtering were applied to reduce false detections and improve object continuity before generating the final bounding boxes. The results shown in Figure 5 demonstrated that the binary U-Net model was capable of producing robust segmentation masks even in cluttered X-ray environments containing overlapping objects and varying object orientations.

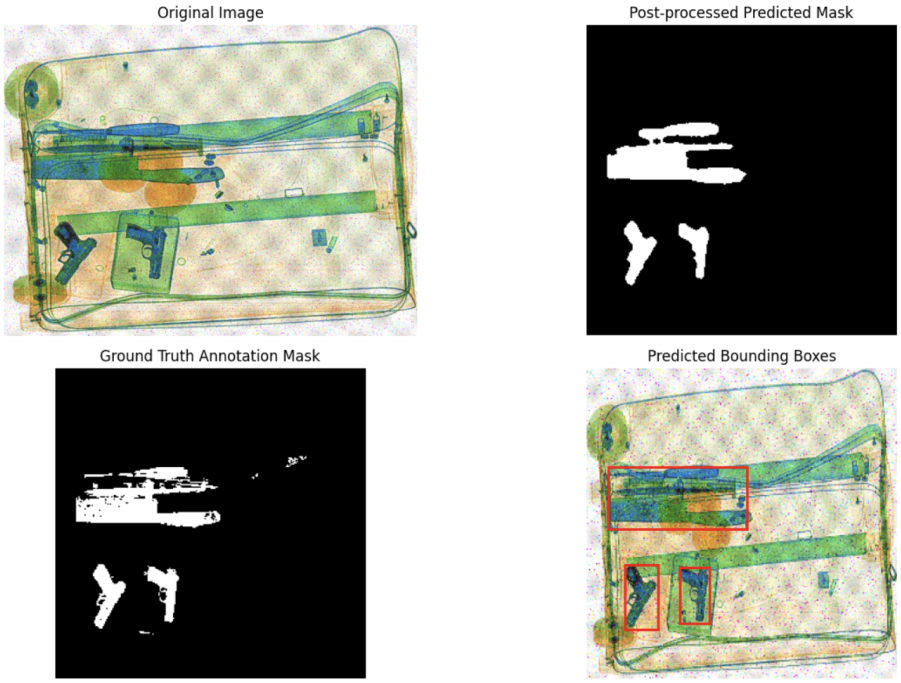


Figure 5: Example of the final U-Net localization results for baggage threat detection. The figure shows the original X-ray image, the predicted post-processed segmentation mask, the ground truth annotation mask, and the final bounding-box localization output generated after connected-component analysis.

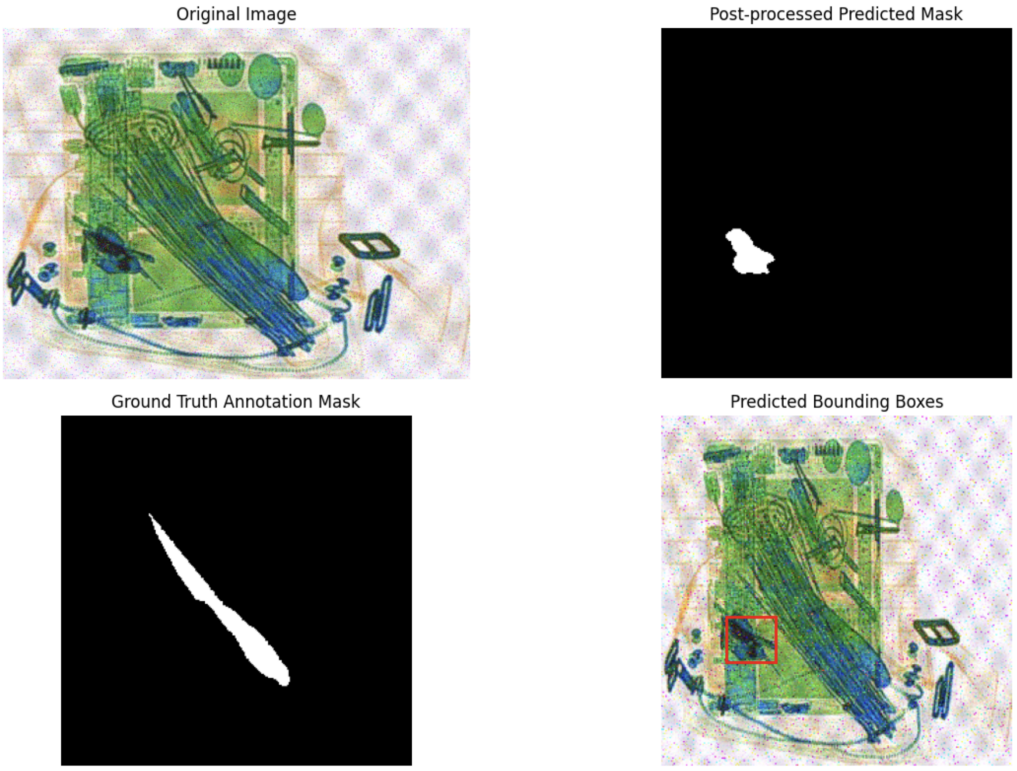


Figure 6: Example of a challenging localization case where the predicted segmentation mask only partially detected the threat object due to cluttered X-ray backgrounds, overlapping structures, and annotation complexity.

Although the final binary U-Net achieved strong localization performance in many cases, several challenging examples still revealed important limitations of the model, as shown in Figure 6. In the example shown above, the U-Net model successfully detected part of the threat object but failed to recover the complete object boundary compared with the ground truth annotation mask. This limitation was mainly caused by heavy background clutter, overlapping baggage contents, weak object contrast, and complex object orientations within the X-ray image. Such cases highlighted the inherent difficulty of segmentation in real-world baggage screening scenarios and emphasized the importance of dataset quality, annotation consistency, and additional model refinement for improving localization robustness.

4 Technical Challenges and Problem Solving

The organization of the dataset was one of my biggest problems. My annotation directory was not organized into a table but had sub-folders organized by class. Some of the code that attempted to use the incorrect annotation format also caused the code to be unable to find the image/mask pairs. To troubleshoot this, I manually reviewed the folder structure, counted the number of files in each folder, and modified the code to search for all of the masks with the same base file name, and the possible extensions .png, .jpg, and .jpeg.

The pixel label data stored in MATLAB also created issues. When working with them, I received errors related to invalid locations as well as missing preprocessing functions. I resolved this issue by changing to use fileDatastore and writing a custom preprocessing function for the masks. I also made sure that each mask was converted to a categorical format that the network expected. This has made me aware that most deep learning code fails due to data not being in the correct format that the training function requires, rather than incorrect network architecture.

One of the main challenges I faced was that the training accuracy was misleading. The MATLAB U-Net showed high accuracy on the pixel level at certain points, but the predicted mask was either all black or a majority was background. Upon further analysis, I learned that the pixel accuracy metric can be misleading for segmentation because the vast majority of pixels in the training image are background pixels. So a model can predict the background everywhere and still obtain a high accuracy metric. Therefore, I began to focus on IoU and Dice score as my primary metrics instead of just accuracy.

Another problem I faced was when the U-Net model would produce masks with false blobs. When the U-Net would find dark baggage regions, it would mistake them for threat objects. To address this, I attempted to use a series of morphological post-processing methods such as opening, closing, filling holes, removing small objects, connected component filtering, area thresholding, aspect ratio checking, and border region rejection, all after discussing the issue with my instructor. While some of these post-processing methods produced improved results, many of the false blobs could not be corrected. This experience helped me understand the difference between post-processing and training errors, as morphology can help reduce noise in poorly learned representations but cannot fully correct them.

When I was debugging my models, I faced another issue regarding the fact that the IoU and Dice scores were showing up as 0, even though they should have been non-zero (because visually, there was an overlap of predicted and annotated masks). I found the problem to be due to how the ground truth (GT) mask was represented. In particular, within the annotation display, object pixels were using 255, but in the evaluation code, they were being evaluated against 1. Once I created two separate versions of the GT mask for evaluation (using a value of 1 for object pixels to suit the evaluation code) and for the display (using a value of 255 for object pixels to suit the annotation display), my metrics made sense. I learned that it is very important to validate that the data types and value ranges of data are correct before having any confidence in the numerical results.

When I began using Google Colab, I encountered a problem with a mismatch between the libraries Keras and PyTorch. Initially, I had used a `.predict()` type call, which failed on my PyTorch model; so, I modified my code to use `model.eval()` and `torch.no_grad()`, then took a Tensor object and streamed it through the model before applying sigmoid and thresholding on the output. Doing this helped me develop an understanding of the fundamental difference between Keras and PyTorch inference workflows.

5 Iterations, Testing, and Refinement

I used an iterative approach to testing. For each image tested, I inspected both the original image and the predicted mask (and annotation mask, as well as bounding boxes) and then adjusted some aspect of preprocessing or post-processing. I used images from the gun, knife, and shuriken folders to determine if the localizer was effective across different shapes and complexities of objects.

One major refinement was to switch from multiclass segmentation to binary segmentation. Multiclass segmentation was an appealing option because it was capable of differentiating between types of objects within images, but it often created ambiguity on object boundaries when annotations were made using binary classes and/or when including several different object classes in an image. Binary segmentation provided one clear objective to assess whether or not the localizer was capable of identifying object pixels regardless of their classification. This helped reduce confusion while training, thus making the localizer more consistent.

Another refinement I made was the initial use of preprocessing techniques. At the beginning of testing, I initially experimented with preprocessing techniques such as median filtering, Wiener filtering, and contrast enhancement to study their effect on segmentation quality. However, after iterative testing, the final U-Net preprocessing pipeline was simplified to grayscale

resizing, mild Gaussian filtering, and normalization because aggressive enhancement techniques often amplified background clutter and reduced localization stability.

I improved my loss function by incorporating both the Dice loss and BCE (cross-entropy) into my Colab implementation. I made this change because the Dice loss works specifically with how well the predicted and ground-truth masks overlap, which is a more accurate reflection of how well the predicted segmentation matches what actually exists, as opposed to just pixel-level matching accuracy. I included data augmentation during training (including horizontal and vertical flips, rotations, and brightness changes) to help improve generalization.

For the purposes of validating my implementation, I used the IoU and Dice scores; I found that they can be more reliable metrics for segmenting objects than measuring the volume of overlap between predicted object areas and the actual ground-truth object; therefore, when interpreting any metric result, it is important to also consider visual representations of the prediction errors to determine if any individual visual representation would lead to misinterpretation of any single value as an individual outcome.

The final binary U-Net model achieved significantly improved localization consistency compared with traditional thresholding and edge-based approaches. The final system successfully localized single and multiple threat objects across various test images while reducing false detections after morphological post-processing and connected-component filtering.

6 Technical Reasoning Behind Final Choices

U-Net is a good choice for localizing objects because there are annotation masks in the dataset, and U-Net is made for pixel-level segmentation. Traditional techniques, like thresholding and edge-based segmentation, are too sensitive to variations, object overlaps, and noise. U-Net enables the system to learn about an object's shape and contextual information from annotated examples.

Binary segmentation was chosen as it was possible with CNN to enable localization within the project scope. The annotations provided were more consistent for localizing an object/background than for specifically labeling multiple object classes. This also eliminates potential misclassification of each object while not interfering with keeping the classification of each object only as a classical classification method.

The final representation of localization is through using bounding boxes, as they can be created from the connected components of the predicted mask. This enables support for multiple objects in an image, as each connected component will provide its own bounding box. Using bounding boxes also prevents us from needing to apply another deep learning model (the use of YOLO or some other object detector), which would take the project farther from its intended direction.

To quantitatively evaluate the localization performance of the proposed U-Net model, Dice similarity coefficient and Intersection over Union (IoU) metrics were computed for multiple test images. These metrics were selected because they provide reliable measurements of segmentation overlap quality between the predicted masks and the ground truth annotations.

Table 2: Localization Performance Evaluation Using Dice Score and IoU

Test Image	**Dice Score**	**IoU**
knife/P00043.png	0.935	0.878
gun/P00096.png	0.864	0.760
shuriken/B0048_0166.png	0.720	0.563
knife/P00184.png	0.901	0.821

The evaluation results demonstrate that the U-Net localization model achieved generally strong segmentation overlap performance across multiple threat categories. Higher Dice and IoU values were obtained for images with clearer object boundaries and less background clutter, while more complex baggage scenes resulted in lower overlap scores due to partial object occlusion and overlapping structures.

Overall, the final system design was not selected based on theoretical assumptions alone, but rather through iterative experimentation, failure analysis, debugging, and comparative evaluation of multiple localization approaches.

7 Lessons Learned

This project helped me to better grasp how classification, localization, object detection, semantic segmentation, and instance segmentation differ from one another. The concepts were often confused prior to this. I used to not distinguish between whether or not U-Net was classifying or localizing an item. From my research and hands-on experience, it became apparent that the output from U-Net is a mask at the pixel level and that classification must be separated from localization unless U-Net is actually designed to produce multi-class segmentation.

When debugging machine learning systems, I have learned that there are many problems found within data paths or mask values, annotation formats, preprocessing choices and metric calculations—in addition to simply changing the model. Thus, I now inspect additional intermediate outputs and do not only consider the final accuracy value.

The other major lesson I learned is that an acceptable-looking preprocessing method may not be productive for training the model. Enhancing contrast will improve image viewing for humans, but it will also enhance the noise and the texture of the background in the image. This made me reevaluate how I view preprocessing—I now assess it based on how it will affect the behavior of the model, rather than how good it looks.

Additionally, I recognised that an open and honest use of AI is fundamental. I created code structure, debugged errors, compared competing design ideas, and wrote reports with AI, but I was cautious when using it for these purposes and conducted testing, eliminated options that didn't work, corrected errors in the code, and validated the results against masks, IoU scores, Dice scores, etc.

8 Possible Improvements and Future Work

In order to enhance the quality of the annotation dataset, the difference between the original images and their corresponding masks may have affected how we calculated IoU and Dice. In addition, I have discovered (in testing) that when a mask does not accurately represent a ground truth mask, the U-Net can still visually appear to perform, but will have decreased scores. Audit of the annotations provided will help increase the reliability of training and evaluation processes.

The inclusion of safe images as true negatives (defined as whole-background masks) would also serve as an improvement. The binary U-Net was primarily trained on threatening images; therefore, it would confuse dark non-threatening areas of baggage (e.g., dark bags) with object types. This additional information should assist the U-Net in learning the types of objects that are dark and do not constitute any type of threat.

The direction we will take in the future is multiclass segmentation if pixel-level class masks are available for all images. This will permit U-Net to create separate gun, knife, and shuriken masks. However, this requires that there are clean annotations and more sufficiently balanced data.

I feel there are better architectures available (such as U-Net++ and an O-net-inspired dual-input architecture) that would improve the model. O-Net was developed specifically for the purpose of detecting dangerous goods in aviation X-ray images. However, I will not implement

the entire O-net right away since it is significantly more complicated than U-Net and requires more training resources to use correctly. Improving U-Net with better augmentation techniques, more negative samples, and improved loss functions is the next step in the progression toward U-Net.

9 Transparent AI Usage

I used ChatGPT as a technical support and brainstorming tool throughout the project. I used it to discuss MATLAB implementations, segmentation strategies, U-Net workflows, debugging ideas, evaluation metrics, and report structuring.

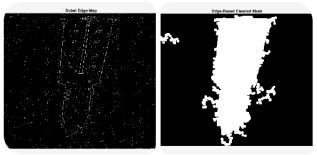
However, I did not accept AI-generated outputs blindly. I tested suggestions experimentally, compared results visually, and rejected methods that did not work effectively on the dataset.

For example, some thresholding and edge-based approaches initially appeared promising theoretically, but after implementation and testing, I observed weak masks and poor generalization. Instead of forcing those methods into the final pipeline, I used the failed results to justify moving toward a learning-based approach.

I also verified AI-generated suggestions by discussing challenges with my instructor and collaborating with my colleagues during the debugging and validation stages.

10 Walkthrough Video Link

Attached is the video link: https://youtu.be/kDQQWtXgnKI



the edge mask is bad
How do I modify it ?

Your edge map is actually good. The **problem is the morphology is too aggressive**.

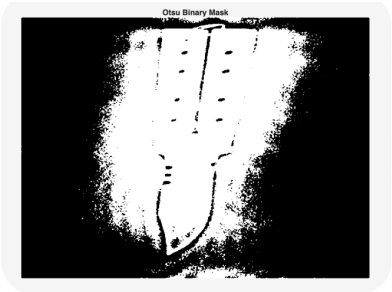
Specifically, this part is making the mask bad:

MATLAB

```
se2 = strel('disk', 6);
bw = imdilate(bw, se2);
bw = imfill(bw, 'holes');
```

The dilation is too large, so background noise connects to the knife.

(a) Sample 1 of ChatGPT help in a specific task.



the otsu method is giving bad mask
What could be the problem?
and How do I know the correct way to solve it?

The problem is **not your code only**. The problem is that **Otsu assumes the image histogram can be separated into two clear groups: object and background**. In your image, the knife and the background have overlapping gray levels, so Otsu selects a threshold that includes a large bright/noisy region around the knife.

This usually happens because:

1. The background is not uniform.
2. The knife boundary is blurred after filtering.
3. Median filtering removed some noise but did not improve object/background contrast.
4. The object is not purely darker or brig than the background.
5. Otsu is global, so one threshold is applied to the whole image.

(b) Sample 2 of ChatGPT help in a specific task.

Figure 7: Examples of AI-assisted debugging and development support used during the baggage threat detection localization implementation and troubleshooting process.